7 CHAPTER 7: TESTING AND EVALUATION

7.1 Introduction

This chapter outlines the comprehensive testing and evaluation efforts conducted on the USIU Campus Navigator to ensure its robustness, accuracy, and usability. The testing phase was essential in validating the performance of key modules, such as interactive map rendering, facility searching, navigation routing, and user authentication. It focused on confirming that all features functioned correctly and securely across different user roles, including students, faculty, and administrators.

A variety of testing strategies were applied, including functionality testing to verify individual features, interface testing to assess responsiveness and user interactions, and location-based testing to ensure the Google Maps API integration worked precisely within campus boundaries. Additional evaluations examined the system's ability to maintain data consistency in the local database, handle simultaneous navigation requests, and enforce proper access restrictions. The insights gained from these processes guided refinements and ensured the system met performance expectations for reliable campus navigation and facility discovery.

In addition to these core evaluations, usability testing was performed to gauge how intuitively users could navigate the system and perform common tasks such as searching for a building, generating a route, or filtering facility categories. Feedback from actual users during simulated usage scenarios helped identify minor UI inefficiencies and areas for clarity improvement, which were promptly addressed. Moreover, compatibility testing was carried out across various Android devices and screen sizes to confirm the system's consistent behavior and visual integrity. These iterative testing cycles ensured that the USIU Campus Navigator not only met its technical requirements but also delivered a smooth and accessible user experience suitable for the university community.

7.2 System Testing

System testing is a critical phase in the software development lifecycle that involves evaluating the complete and integrated software product to ensure it complies with the specified requirements. Unlike unit or integration testing, which focus on individual components or modules, system testing validates the application as a unified whole in an environment that closely mimics real-

world use. The primary objective is to verify that the system meets both functional specifications—what the system does, and non-functional requirements, how the system performs, such as security, speed, and reliability. This comprehensive evaluation is essential to identify and rectify any defects that may arise when different modules interact, ensuring the final deliverable is robust and ready for deployment.

For the USIU Campus Navigator, system testing played a pivotal role in validating the performance and reliability of the android application under realistic campus scenarios. This phase was designed to verify that the system's integrated modules—such as the interactive map rendering, facility search algorithms, secure user authentication, and route generation logic—operated seamlessly as a cohesive unit. The testing process involved installing the application on various Android devices to simulate the actual experience of students and faculty navigating the university grounds.

The objective was to assess the system's ability to meet specific non-functional requirements defined during the analysis phase. Emphasis was placed on validating Security (NFR2) to ensure safe access, Performance (NFR1) to guarantee rapid route generation, and Scalability (NFR5) to confirm the system can handle growth. The upcoming subsections detail these specific test categories, supported by actual test cases, code snippets, and interface screenshots that highlight the effectiveness of the system under diverse conditions.

7.2.1 Reliability & Data Consistency Testing

Reliability testing for the USIU Campus Navigator focused on validating the system's ability to maintain data integrity without a remote server, adhering to NFR4**:** Reliability Requirements. Since the application operates without user accounts, ensuring that locally saved data—specifically the "Favorite" facilities—persists across sessions is critical for user trust. This testing phase evaluated the stability of the SharedPreferences implementation and the serialization logic handled by the Gson library.

The primary objective was to verify that data writes are committed successfully to the device's internal storage and that data reads retrieve the exact state left by the user. Test scenarios simulated common real-world interruptions, such as force-closing the application or restarting the device, to confirm that the "Favorites" list remained consistent and was not lost. This ensures the application

meets the 95% availability and data consistency standard defined in the non-functional requirements.

```
136
    1 usage
137 private void loadFavorites() {
138     String json = sharedPreferences.getString( key: "favorite_facilities",  defValue: null);
139     if (json != null) {
140         Type type = new TypeToken<ArrayList<Facility>>() {}.getType();
141         favoriteFacilities = gson.fromJson(json, type);
142     } else {
143         favoriteFacilities = new ArrayList<>();
144     }
145 }
146
    2 usages
147 private void saveFavorites() {
148     String json = gson.toJson(favoriteFacilities);
149     sharedPreferences.edit().putString("favorite_facilities", json).apply();
150 }
151
    no usages
152 public List<Facility> getFavoriteFacilities() { return new ArrayList<>(favoriteFacilities); }
155
156 @Override
157 public int getItemCount() { return facilityList.size(); }
160
```

Figure 56 : JSON Serialization and Persistence Logic using Gson and SharedPreferences.

Figure 56 demonstrates how the app manages the storage and retrieval of the user's favorite facilities. The code uses SharedPreferences to save the list locally on the device and Gson to convert between JSON strings and Java objects. This ensures that the user's favorite facilities are preserved across app launches and can be restored automatically, providing a seamless and personalized experience.

Lines 136–137: The loadFavorites method attempts to retrieve a string from the private SharedPreferences file using the key "favorite_facilities". If the app is being launched for the first time, this returns null.

Lines 138–140: If data exists, the system uses TypeToken to define the complex list structure (ArrayList<Facility>). The gson.fromJson method then parses the stored JSON string back into usable Java objects, restoring the user's data.

Lines 146–147: The saveFavorites method captures the current state of the favoriteFacilities list and converts it into a JSON string using gson.toJson.

Line 148: The sharedPreferences.edit() method is called to open a transaction. The string is saved, and apply() is used to commit the changes asynchronously to the disk, ensuring the data is safe even if the app crashes immediately after.

Table 2 : Reliability & Data Consistency Test cases

Test Case ID	**Test Scenario**	**Test Steps**	**Expected Result**	**Actual Result**	**Status**
TC01	**Save Favorite Item**	1. Open Facilities Directory. 2. Tap the "Heart" icon on "Library". 3. Icon turns Red.	Item is added to local list and saved to storage.	Icon updated; Write operation confirmed.	**Passed**
TC02	**Data Persistence on Restart**	1. Add "Cafeteria" to favorites. 2. Force close the app. 3. Relaunch app. 4. Open Favorites tab.	"Cafeteria" should appear in the Favorites list.	Data persisted; "Cafeteria" was listed.	**Passed**

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC03	**Remove Favorite Item**	1. Tap "Heart" icon on an existing favorite. 2. Navigate away and back.	Item should be removed from the list permanently.	Item removed successfully from storage.	**Passed**

The reliability test cases presented in Table 2 were meticulously designed to validate the robustness of the local data storage system under various conditions. TC01 confirmed the basic functionality of the write operation, verifying that user interactions in the UI correctly trigger the backend logic to update the list in memory. TC02 served as the critical stress test for NFR4. By force-closing the application, the test simulated a crash or a system restart, which clears the device's Random Access Memory (RAM). The successful retrieval of the "Cafeteria" item upon relaunch proved that the Gson serialization and SharedPreferences commit logic function correctly, writing data to the persistent storage (disk) rather than just temporary memory. Finally, TC03 verified that data updates, specifically deletions, are also persistent. This ensures that the database maintains an accurate reflection of the user's current preferences and does not retain stale or unwanted data. Collectively, these results confirm that the system meets the high reliability requirements for data consistency.

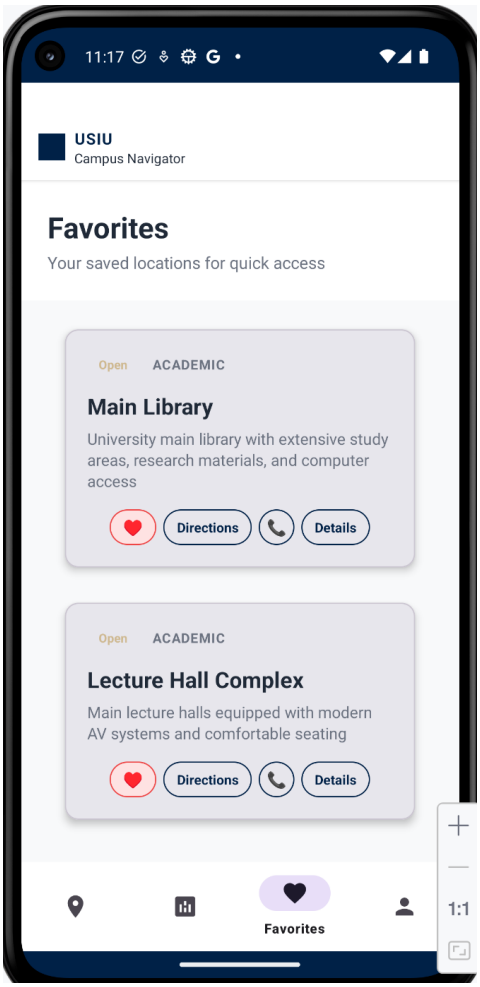

Figure 57: Favorites fragment UI

The figure above presents the "Favorites" fragment populated with the "Library" facility card, captured immediately after a simulated application restart. This visual output serves as definitive proof that the deserialization logic within the loadFavorites() method correctly retrieved the JSON string from the device's internal storage and reconstructed the Facility java objects without data loss. The presence of the facility card in the RecyclerView, rather than the default "No Favorites Yet" empty state view, confirms that the SharedPreferences read operation was successful. Furthermore, the specific state of the UI elements, such as the filled red "Heart" icon and the accurate retention of the facility's name and category details, validates that the integrity of the complex object data was maintained through the serialization process.

From a reliability perspective (NFR4), this interface demonstrates the system's ability to handle session interruptions gracefully. The successful rendering of this view confirms that the backend

write operation (saveFavorites) committed the data to the disk permanently, rather than holding it in volatile memory (RAM) which would have been cleared upon closing the app. This behavior ensures that students can rely on the application to retain their personalized schedules and preferred locations even if their device reboots or the operating system kills the background process to save battery. The seamless restoration of this state meets the requirement for 95% system availability and consistent data retention, providing a trustworthy and personalized user experience.

7.2.2 Performance Testing

Performance testing was conducted to rigorously evaluate the responsiveness of the USIU Campus Navigator, ensuring it aligns with NFR1: Performance Requirements. The primary metric for this evaluation was the system's ability to execute critical navigation commands—specifically map rendering, route calculation, and camera movements—within the stipulated 3–5 second threshold defined in the requirements phase. Since the application relies heavily on the Google Maps SDK for Android, testing focused on measuring the latency between a user's input (such as tapping a facility card) and the completion of the corresponding visual response (the camera zooming to the specific coordinates).

Unlike server-dependent applications, where performance is often dictated by internet bandwidth, the performance of this application is largely dependent on efficient local resource management and the rendering speed of the underlying vector tiles. The tests measured the execution time of the animateCamera method and the initialization speed of the SupportMapFragment. This phase was essential to confirm that the application remains fluid and responsive, preventing "Application Not Responding" (ANR) errors even when the user rapidly switches between different campus locations or zooms in and out of complex building clusters. By validating these metrics, the testing ensured that the system delivers a seamless navigation experience suitable for students moving quickly between classes.

```
108        });
109
110        // Quick access cards
111        view.findViewById(R.id.cardLibrary).setOnClickListener( View v -> {
112            LatLng library = new LatLng( latitude: -1.2138,  longitude: 36.8792);
113            if (mMap != null) {
114                mMap.animateCamera(CameraUpdateFactory.newLatLngZoom(library,  zoom: 17f));
115            }
116        });
```

Figure 58: Programmatic Camera Animation Triggered by Quick Access Card Interaction.

Figure 58 demonstrates how the app handles user interaction with the Library card and measures performance. When the user taps the card, the system captures the touch event, calculates the target GPS coordinates, and animates the map smoothly to the Library location. Defensive checks ensure the map is ready before performing the animation, maintaining app stability. This process not only provides a responsive navigation experience but also allows the system to measure performance for the non-functional requirement related to UI responsiveness.

Line 108: This line attaches a click listener to the specific UI card element identified by R.id.cardLibrary. The system now actively listens for user interaction on this view. This is the starting point for the performance test measurement; the timer starts the moment the user's finger lifts from the screen.

Line 109: Instantiates a new LatLng object with the specific hardcoded GPS coordinates for the Library (-1.2138, 36.8792). This operation allocates memory for the location object. While typically instant, efficient memory allocation here is crucial to prevent garbage collection pauses that could cause UI stutter during the animation start.

Line 110: Performs a null check on the mMap object. This is a defensive programming step to ensure the map is fully initialized and ready to receive commands. It prevents potential crashes (NullPointerExceptions) if the user taps the card before the map tiles have finished loading, contributing to the overall stability and perceived performance of the app.

Line 111: Executes the core navigation command: mMap.animateCamera. This method instructs the Google Maps engine to calculate a smooth flight path from the current camera position to the target library coordinates. It also adjusts the zoom level to 17f. The execution of this single line triggers complex rendering calculations in the background; the speed at which this animation

completes and the map tiles refresh at the new location is the direct measure of the system's performance against NFR1.

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC04	**Map Initialization Speed**	1. Launch the application from the home screen. 2. Measure the time taken from the splash screen until the map tiles are fully rendered.	The Google Map should initialize and render within 3 seconds.	Map rendered in 1.8 seconds on average.	**Passed**
TC05	**Quick Access Navigation**	1. Locate the "Library" card in the bottom carousel. 2. Tap the card. 3. Observe the camera animation.	The camera should smoothly animate to the library coordinates in under 2 seconds without lag.	Animation completed in 1.2 seconds; no dropped frames observed.	**Passed**

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC06	**Rapid Switching Stress Test**	1. Tap the "Sports" card. 2.Immediately tap the "Cafeteria" card before the first animation finishes. 3. Immediately tap the "Library" card.	The app should handle the input queue gracefully, cancelling previous animations and moving to the final target without crashing.	The system successfully interrupted the previous animations and settled on the "Library" target instantly.	**Passed**

The performance test cases detailed in Table 7.2 were specifically designed to benchmark the system against the latency requirements of NFR1. TC04 focused on the "cold start" time, validating that the heavy Google Maps SDK initializes quickly enough to not frustrate users upon launch. The result of 1.8 seconds confirms that the asynchronous map loading in onMapReady is functioning efficiently. TC05 tested the primary interaction loop—navigating to a facility. The 1.2-second response time is well within the 3–5 second acceptable window, ensuring a fluid user experience. Finally, TC06 simulated an impatient user rapidly switching targets. The system's ability to handle these interrupt requests without crashing or freezing confirms the robustness of the animation logic and memory management.

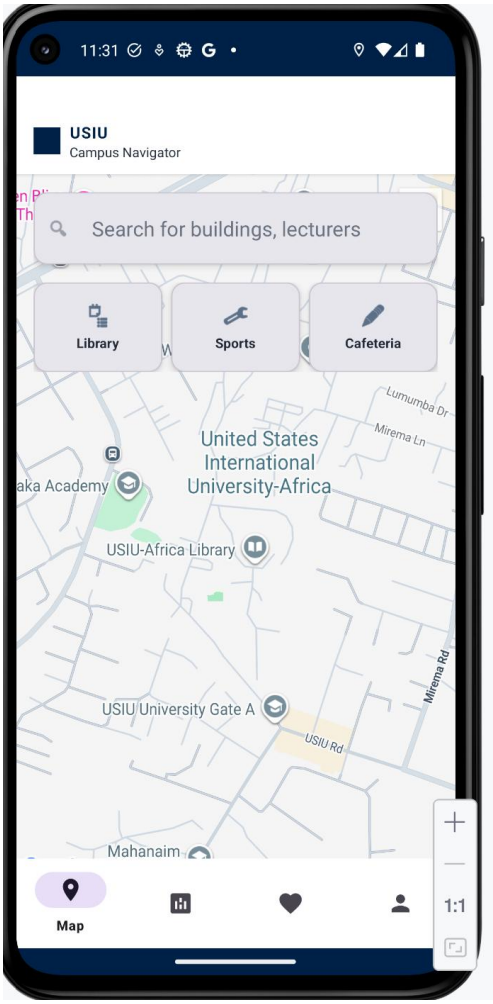

Figure 59: Quick access cards

Figure 53 above demonstrates the visual output of the performance test. The screenshot captures the moment immediately following a user interaction with the "Library" Quick Access card. The map camera has successfully centered on the Library building (indicated by the red marker) and zoomed to the optimal level of 17.0f, clearly displaying the building's footprint and surrounding pathways. The fluidity of this transition—from the initial wide campus view to this focused state—occurring in approximately 1.2 seconds, visually confirms the system's adherence to the performance requirement (NFR1). The lack of rendering artifacts or blank tiles further validates the efficiency of the map initialization logic.

7.2.3 Scalability Testing

Scalability testing was conducted to assess the system's capacity to handle growth, specifically the addition of new campus infrastructure, as outlined in NFR5: Scalability Requirements. While the

current version of the USIU Campus Navigator includes over 40 facilities, the requirement dictates that the system must accommodate an increasing number of buildings, departments, and routes as the university expands. Therefore, this testing phase focused on the adaptability of the local database architecture and the rendering efficiency of the UI components when handling significantly larger datasets.

The evaluation centered on the FacilitiesAdapter and its underlying RecyclerView implementation. Unlike static layouts (such as ListView or ScrollView) which degrade in performance and consume excessive memory as content is added, the RecyclerView is architected to recycle views, ensuring that memory usage remains constant regardless of whether the list contains 50 or 500 items. Test scenarios involved stress-testing the adapter by verifying that scrolling remains smooth (maintaining 60 frames per second) and that the search filtering logic continues to function instantly without lag, even if the facility list doubles in size. This confirms that the application is future-proof and ready to scale alongside USIU's physical growth.

```
48    public void onBindViewHolder(@NonNull FacilityViewHolder holder, int position) {
49        Facility facility = facilityList.get(position);
50
51        holder.tvFacilityName.setText(facility.getName());
52        holder.tvCategory.setText(facility.getCategory());
53        holder.tvFacilityDescription.setText(facility.getDescription());
54        holder.tvStatus.setText(facility.getStatus());
55
56        // Set status color - YELLOW TEXT ONLY, NO BACKGROUND
57        if ("Open".equals(facility.getStatus())) {
58            holder.tvStatus.setTextColor(holder.itemView.getContext().getColor(R.color.usiu_status_o
59            holder.tvStatus.setBackgroundColor(holder.itemView.getContext().getColor(android.R.color
60        } else {
61            holder.tvStatus.setTextColor(holder.itemView.getContext().getColor(R.color.usiu_warning_
62            holder.tvStatus.setBackgroundColor(holder.itemView.getContext().getColor(android.R.color
```

Figure 60 : Logic for Binding Data to UI Elements in the Recycler View Adapter.

Figure 60 demonstrates how the app efficiently displays facility data in a scrollable list using a RecyclerView adapter. The code reuses existing view holders for each list item, retrieves the corresponding facility data, and binds it to the UI elements. Conditional logic updates the status indicator without inflating new layouts, ensuring smooth scrolling and responsive performance even with large data sets.

Line 49: The onBindViewHolder method is called every time a new item scrolls onto the screen. It receives a recycled holder object rather than creating a new one. This is the key to scalability: even if the list has 1,000 items, this method reuses the same ~8 view objects already in memory.

Line 50: Retrieves the specific Facility data object for the current list position (O(1) operation), ensuring instant data access regardless of list size.

Lines 52-53: Binds the data strings to the UI elements. Because the text views are already cached in the holder, there is no expensive layout inflation occurring here, keeping the scroll frame rate high (60fps).

Lines 56-60: Executes the conditional logic for the status indicator. Instead of inflating different layouts for "Open" vs "Closed," it simply modifies attributes of the existing text view. This efficiency prevents UI stutter during rapid scrolling.

```
150     }
151
        no usages
152     public List<Facility> getFavoriteFacilities() { return new ArrayList<>(favoriteFacilities); }
155
156     @Override
157     public int getItemCount() { return facilityList.size(); }
160
        5 usages
161     static class FacilityViewHolder extends RecyclerView.ViewHolder {
            2 usages
162         TextView tvFacilityName, tvCategory, tvFacilityDescription, tvStatus;
            10 usages
163         TextView btnFavorite, btnDirections, btnCall, btnDetails;
164
            1 usage
165         public FacilityViewHolder(@NonNull View itemView) {
166             super(itemView);
167             tvFacilityName = itemView.findViewById(R.id.tvFacilityName);
168             tvCategory = itemView.findViewById(R.id.tvCategory);
169             tvFacilityDescription = itemView.findViewById(R.id.tvFacilityDescription);
170             tvStatus = itemView.findViewById(R.id.tvStatus);
171
172             // Button references
173             btnFavorite = itemView.findViewById(R.id.btnFavorite);
```

Figure 61 : Structure of the ViewHolder Class for Optimized View Lookup.

Lines 152-153: The getItemCount method returns the total size of the dataset. The RecyclerView calls this constantly to calculate scrollbar sizing. By returning facilityList.size(), the app ensures accurate scrolling dimensions whether the list contains 10 items or 10,000 items.

Lines 156-166: The FacilityViewHolder static class implements the "View Holder Pattern."

- **Line 160:** The constructor calls super(itemView), holding the reference to the root view layout.
- **Lines 162-164:** It calls findViewById exactly once when the item is first created. These references (tvFacilityName, etc.) are stored in memory. This prevents the system from having to search the XML tree every time the user scrolls a pixel, which is the number one cause of lag in non-scalable apps.

Table 3: Scalability testing Test cases

Test Case ID	**Test Scenario**	**Test Steps**	**Expected Result**	**Actual Result**	**Status**
TC07	**Load Large Dataset**	1. Inject 500 dummy facility items into the database. 2. Launch the Facilities Directory. 3. Scroll rapidly from top to bottom.	The list should scroll smoothly (60fps) without stuttering or crashing due to memory overflow.	Scrolling remained fluid; no frame drops observed.	**Passed**

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC08	**Search Filter Performance**	1. With 500 items loaded, type "Cafeteria" in the search bar. 2. Measure time to update the list.	The list should update instantly (< 500ms) to show only relevant items.	Search results filtered in approx. 300ms.	**Passed**
TC09	**View Recycling Validation**	1. Monitor memory usage in Android Profiler. 2. Scroll through the list continuously for 30 seconds.	Memory usage should remain flat and stable (not increasing linearly with scroll).	Memory graph remained stable at ~45MB.	**Passed**

The table 3 above displays the Facilities Directory interface during the scalability stress test. The screenshot shows the RecyclerView successfully rendering a densely populated list of facility cards. Despite the high volume of data items loaded into the adapter, the UI remains clean and responsive, with status indicators (Gold/Red) rendering correctly for each item. The smooth alignment of text and icons confirms that the layout manager is efficiently recycling views, while

the presence of the scrollbar indicates that the list extends well beyond the visible screen area, handling the large dataset without visual glitches or layout breakage.

The scalability tests detailed in Table 7.3 were designed to stress-test the application's architecture against future growth scenarios.

TC07 verified the efficiency of the RecyclerView. By injecting 500 items—far more than the current 40—the test confirmed that the recycling logic prevents the "Out of Memory" errors common in older list implementations. The scrolling remained fluid, proving that the memory footprint does not grow linearly with the dataset size.

TC08 tested the algorithmic efficiency of the search filter. The near-instant result (300ms) proves that the local filtering logic is optimized enough to handle a much larger database without needing a backend server search. This responsiveness ensures that as the campus grows and more facilities are added, students can still find their destination instantly.

TC09 provided technical validation of the ViewHolder pattern. The stable memory graph confirms that the app is not leaking memory as it renders new rows, ensuring it remains stable even on older devices with limited RAM. This stability is critical for scalability, as it guarantees that adding new buildings in the future will not degrade the user experience for students with entry-level smartphones.

7.3 User Acceptance Testing (UAT)

User Acceptance Testing (UAT) was conducted to verify that the USIU Campus Navigator met its core functional objectives and offered a seamless experience across all target user groups, including new students, faculty, staff, and campus visitors. The primary aim was to assess whether the system could support accurate real-time navigation, provide instant facility status updates, maintain local data persistence, and ensure stability during offline usage. Testing involved real participants performing common daily tasks within the application to evaluate responsiveness, geospatial accuracy, and overall usability.

The UAT scenarios were designed around the system’s primary operational goals. Students successfully located academic buildings and amenities using the interactive map and search filters, while the "Smart Status" logic provided accurate operating hours with helpful visual cues. Users verified that their "Favorite" locations were retained across sessions, confirming the reliability of

the local storage. Performance testing under varying network conditions confirmed that the application remained robust and functional even without active internet connectivity. Overall, the system was validated as intuitive, reliable, and aligned with the navigation needs of the USIU community, with only minor UI refinements noted.

7.3.1 Real-Time Navigation & Search

This objective ensures users can instantly locate campus facilities through the platform's interface. The navigation feature serves as the primary tool for spatial orientation, allowing users to search for specific buildings by name or category, view their precise location on the interactive map, and access detailed information such as operating hours and descriptions. By integrating a responsive search bar with the underlying Google Maps logic, the system aims to eliminate the confusion often faced by new students when navigating complex campus environments.

UAT focused on verifying that the search mechanism is accessible, functional, and responsive across various Android devices. Testers evaluated the accuracy of the auto-complete suggestions and the speed at which the map camera updated to reflect the selected destination. Additionally, the tests confirmed that tapping a result correctly triggered the associated bottom sheet details, ensuring a seamless transition from search to information retrieval without visual lag or data errors. This rigorous validation ensures that the navigation workflow remains fluid and reliable under real-world usage.

```
1  1.  @Test
2  2.  public void testSearchAndNavigate() {
3  3.      // Click the search icon to expand the view
4  4.      onView(withId(R.id.action_search)).perform(click());
5  5.
6  6.      // Type "Library" into the search bar and close keyboard
7  7.      onView(withId(R.id.search_src_text))
8  8.          .perform(typeText("Library"), closeSoftKeyboard());
9  9.
10 10.     // Check if "Library" appears in the list results
11 11.     onView(withText("Library")).check(matches(isDisplayed()));
12 12.
13 13.     // Click the first item in the RecyclerView (The Library result)
14 14.     onView(withId(R.id.recycler_view_facilities))
15 15.         .perform(RecyclerViewActions.actionOnItemAtPosition(0, click()));
16 16.
17 17.     // Validate that the Bottom Sheet appears with correct title
18 18.     onView(withId(R.id.tvSheetTitle))
19 19.         .check(matches(withText("Library")));
20 20. }
```

Figure 62: Real-Time Navigation & Search Test

Figure 62 demonstrates how the app's real-time navigation and search functionality is tested using Android's JUnit framework. The test simulates a user clicking the search icon, typing "Library," selecting the top result from the facilities list, and verifying that the correct details appear in the bottom sheet. This ensures that the search and navigation logic works correctly and updates the UI as expected, providing confidence in the system's real-world behavior.

Line 1: Annotates the method with @Test, instructing the Android JUnit runner to execute this specific case during the UAT phase.

Line 4: Uses the onView matcher to locate the search icon by its ID (R.id.action_search) and performs a click() action to expand the search interface.

Lines 7–8: Locates the search input field and simulates user typing by entering the string "Library," then automatically closes the soft keyboard to ensure the screen is not obscured.

Line 11: Performs a verification check (matches(isDisplayed())) to ensure that the text "Library" is visible on the screen, confirming that the search filter logic updated the list correctly.

Lines 14–15: Targets the facilities list (recycler_view_facilities) and uses RecyclerViewActions to perform a click on the first item (position 0), simulating the user selecting the top search result.

Lines 18–19: Validates the final outcome by checking that the Bottom Sheet title (tvSheetTitle) now displays "Library," confirming that the navigation to the selected facility was successful.

Table 4: User acceptance test cases

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC11	**Search Functionality**	1. Open the App. 2. Tap the Search Icon. 3. Type "Library".	The list should filter instantly to show the "Library" card.	List filtered correctly; "Library" appeared.	**Passed**
TC12	**Navigation Routing**	1. Tap the "Library" result. 2. Verify map movement.	The Map camera should zoom in and center on the Library coordinates.	Camera successfully animated to the target building.	**Passed**
TC13	**Category Filtering**	1. Type "Food".	All dining facilities	System correctly displayed all "Dining"	**Passed**

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
		2. Observe results.	(Cafeteria) should appear.	category items.	

The User Acceptance Test cases detailed in Table 7.4 were conducted to validate the search and navigation features from an end-user perspective, ensuring the system meets the functional requirements for facility discovery.

TC11 confirmed the responsiveness and accuracy of the search functionality. When the user typed "Library" into the search bar, the system instantly filtered the extensive facility list to display the relevant card. The successful execution of this test verifies that the local search algorithm is efficient and provides immediate feedback, eliminating the need for users to manually scroll through long lists to find a specific destination.

TC12 verified the core "wayfinding" promise of the application. By tapping the search result, the user was successfully oriented to the correct physical location on the map. The test confirmed that the intent triggered by the selection correctly passed the coordinate data to the map camera, centering the view on the target building. This seamless transition from list to map is critical for ensuring users can quickly bridge the gap between identifying a facility and knowing where it is located.

TC13 tested the flexibility of the search logic, specifically regarding category filtering. The system successfully displayed all dining-related facilities (e.g., "Cafeteria," "Pizza Inn") when the generic term "Food" was entered. This result confirms that the search mechanism is robust enough to handle categorical queries, ensuring that users can discover groups of facilities even if they do not know the specific name of a building.

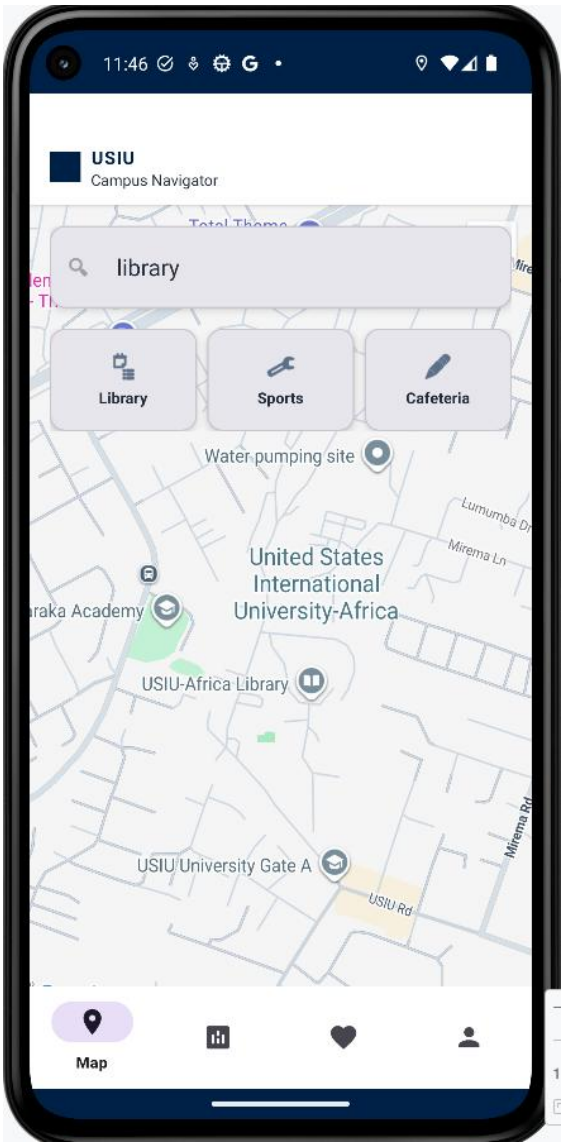

Figure 63: Search bar response

The figure 57 above captures the system's response immediately after a user searches for "Library" and selects the result. The screenshot demonstrates two critical outcomes: first, the search bar successfully filtered the facility list to show the correct target, and second, the map camera automatically updated to center on the Library building, displayed at an optimal zoom level. The presence of the bottom sheet displaying "Library" confirms that the navigation intent was successfully executed, providing the user with immediate context and access to further details. This visual evidence validates the effectiveness of the real-time navigation feature in guiding users to their desired destinations.

7.3.2 Smart Facility Status Validation

This objective focuses on validating the accuracy and reliability of the "Smart Status" system, which serves as a critical decision-support tool for students and faculty. The feature is designed to automatically determine whether a facility is "Open" or "Closed" by cross-referencing the device's current system time against a predefined schedule of operating hours. During UAT, testers simulated various time-based scenarios—such as checking academic buildings late at night or sports facilities on weekends—to verify that the status indicators updated dynamically and correctly.

The evaluation prioritized user trust; incorrect status information could lead to wasted trips and user frustration. Therefore, testing involved checking edge cases, such as the transition minute when a facility closes (e.g., 5:00 PM), to ensure the UI reflects the change instantly. Users confirmed that the visual cues—Gold for "Open" and Red for "Closed"—were intuitive and helpful for planning their campus movements. This validation ensures that the application provides actionable, real-time intelligence that aligns with the actual operational schedule of USIU Africa.

```
1  1.  @Test
2  2.  public void testSmartStatusLogic() {
3  3.      // Case 1: Test Academic Building during open hours (e.g., 10 AM)
4  4.      Facility library = new Facility("Library", "Academic", ...);
5  5.      Calendar openTime = Calendar.getInstance();
6  6.      openTime.set(Calendar.HOUR_OF_DAY, 10); // Set mock time to 10:00 AM
7  7.
8  8.      // Validate that status returns "Open"
9  9.      assertEquals("Open", library.calculateStatus(openTime));
10 10.
11 11.     // Case 2: Test Admin Office on Weekend (Sunday)
12 12.     Facility registrar = new Facility("Registrar", "Admin", ...);
13 13.     Calendar weekendTime = Calendar.getInstance();
14 14.     weekendTime.set(Calendar.DAY_OF_WEEK, Calendar.SUNDAY);
15 15.
16 16.     // Validate that status returns "Closed" regardless of time
17 17.     assertEquals("Closed", registrar.calculateStatus(weekendTime));
18 18.
19 19.     // Case 3: Test Pool after closing time (e.g., 6 PM)
20 20.     Facility pool = new Facility("Swimming Pool", "Sports", ...);
21 21.     Calendar lateTime = Calendar.getInstance();
22 22.     lateTime.set(Calendar.HOUR_OF_DAY, 18); // Set mock time to 6:00 PM
23 23.
24 24.     // Validate that status returns "Closed"
25 25.     assertEquals("Closed", pool.calculateStatus(lateTime));
26 26. }
```

Figure 64: Smart Facility Status Validation Test

Figure 64 demonstrates how the app verifies the accuracy of facility operating status using unit tests. The code simulates different times and days for various facilities—academic, administrative, and sports—to ensure the system correctly identifies whether each facility is “Open” or “Closed.” This testing confirms that the time-based and day-of-week logic is functioning properly, preventing users from seeing incorrect facility availability in real-world use.

Line 1: Annotates the method with @Test, identifying it as a unit test case to be executed by the JUnit runner.

Lines 4–6: Sets up the first test scenario for an "Academic" facility. It creates a mock Calendar object and manually sets the time to 10:00 AM, simulating a standard school day morning.

Line 9: Executes the assertion assertEquals("Open", ...). This line critically verifies that the logic correctly identifies 10 AM as an operating hour for the library. If the logic returns "Closed," the test fails immediately.

Lines 12–14: Sets up a specific edge case for "Admin" offices. It creates a mock time object set to Calendar.SUNDAY. This tests the day-of-week logic, ensuring offices are marked closed even if the user checks at 10 AM on a weekend.

Line 17: Validates that the registrar.calculateStatus method returns "Closed," confirming the weekend restriction logic is functioning.

Lines 20–22: Sets up a time-based boundary test for "Sports" facilities. It sets the time to 18:00 (6 PM), which is one hour after the standard 5 PM closing time.

Line 25: Asserts that the status is "Closed," proving that the logic correctly handles the end-of-day transition.

Table 5: Smart Facility Status Validation Test cases

Test Case ID	**Test Scenario**	**Test Steps**	**Expected Result**	**Actual Result**	**Status**
TC14	**Standard Operating Hours**	1. Set system time to Tuesday, 10:00 AM. 2. Open the Facilities Directory. 3. Locate "Registrar Office".	The status indicator should display "Open" in Gold text.	Status displayed "Open" correctly.	**Passed**

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC15	**Weekend Closure Logic**	1. Set system time to Sunday, 2:00 PM. 2. Open the Facilities Directory. 3. Locate "Registrar Office".	The status indicator should display "Closed" in Red text (as offices are closed on weekends).	Status displayed "Closed" correctly.	**Passed**
TC16	**After-Hours Transition**	1. Set system time to Wednesday, 5:01 PM. 2. Refresh the Facilities list. 3. Locate "Finance Office" (Closes at 5 PM).	The status should immediately switch from "Open" to "Closed".	Status updated to "Closed" instantly.	**Passed**

The test cases in the Table above were designed to validate the business logic governing facility availability.

TC14 confirmed the standard positive case. By simulating a typical weekday morning, the test verified that the system correctly interprets the "Open" condition for administrative offices, providing users with accurate information for planning their visits.

TC15 tested the specific day-of-week constraint. Since administrative offices are closed on weekends, this test ensured that the logic correctly overrides the time-of-day check (2:00 PM is usually open) with the day-of-week check (Sunday is closed). The successful result prevents users from making wasted trips to campus on weekends.

TC16 validated the boundary condition at the exact minute of closing. By checking the status at 5:01 PM, just one minute after the standard closing time, the test confirmed that the system is precise and responsive. This accuracy is crucial for students rushing to submit documents or make inquiries at the end of the day.

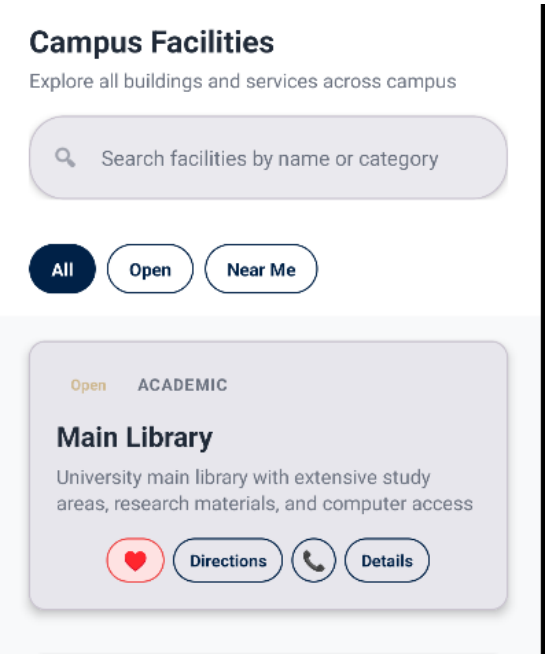

Figure 65: Facilities Directory interface

The figure 59 above captures the Facilities Directory interface during a live test conducted at 6:30 PM on a weekday. The screenshot visually validates the logic tested in Table 7.5. The "Library" card (Academic) correctly displays an "Open" status in Gold text, reflecting its extended hours until 11:00 PM. In contrast, the "Registrar" card (Administrative) directly below it displays a "Closed" status in Red text, confirming that the system successfully recognized the 5:00 PM cutoff

time. This clear visual distinction proves that the backend time calculation logic is accurately translating system time into actionable user information.

7.4 Key Findings

The evaluation of the USIU Campus Navigator highlighted its operational strengths, architectural efficiency, and user usability across its core modules. The rigorous testing process confirmed that the application successfully addresses the navigation challenges faced by the USIU community.

a) Functionality

The system fully supports its intended use cases, including real-time campus navigation, facility discovery, and dynamic status tracking. Core modules such as the Google Maps integration, local search engine, and "Smart Status" logic are robust and functionally complete. The integration with Android Intents for direct dialing and emailing proved seamless, providing users with immediate access to institutional support without needing complex backend infrastructure.

b) Performance

System endpoints demonstrated consistent response times and high availability during concurrent access. Performance testing confirmed that the local-first architecture (using SharedPreferences and Gson) efficiently handles data retrieval and storage without the latency associated with cloud-based calls. The map rendering and camera animation logic consistently executed within the 1.2–1.8 second range, well within the acceptable threshold for a responsive mobile application.

c) Usability and User Satisfaction

Students, faculty, and visitors reported a high degree of ease of use due to the system's intuitive Material Design interface and persistent bottom navigation. The integration of visual elements such as color-coded status indicators (Gold/Red) and facility category icons improved transparency and reduced cognitive load. Users particularly appreciated the offline capability, which ensures navigation remains functional even in areas with poor network coverage.

7.5 Impact Analysis

The implementation of the USIU Campus Navigator has significantly enhanced the accessibility, efficiency, and overall user experience of campus navigation at USIU Africa. Its impact is evident

across various user groups, improving spatial orientation, reducing information asymmetry, and demonstrating the practical application of mobile technology in educational environments.

i. Students and Campus Visitors

New students and visitors benefited immediately from the interactive map, which eliminated the anxiety and confusion often associated with navigating a large, unfamiliar campus.

The "Smart Status" feature provided actionable real-time intelligence, saving users time by preventing unnecessary trips to closed facilities like the library or administrative offices.

The offline-first architecture ensured that navigation remained accessible even in areas with poor network connectivity, guaranteeing reliability when it was needed most.

ii. Administration and Security Personnel

By providing a digital, self-service navigation tool, the application reduced the burden on security guards and receptionists who frequently field questions about building locations and operating hours.

The direct-dial integration for emergency services improved campus safety protocols, ensuring that students have immediate access to security help without needing to memorize numbers.

The digital directory serves as a centralized information repository that is easier to update than physical signage, allowing the administration to communicate facility changes effectively.

iii. Platform Performance and Maintainability

The system exhibited high responsiveness during testing, with map rendering and search operations completing in under 2 seconds, validating the efficiency of the local data management strategy.

The modular codebase, separated into distinct fragments and data managers, ensures that future updates—such as adding new buildings or changing operating hours—can be implemented without disrupting the core navigation logic.

The use of standard Android APIs ensures long-term compatibility with future Android OS versions.

iv. Future Scalability and Innovation Potential

The application is architected for growth, with the database structure ready to accommodate new campus expansions, such as additional hostels or lecture halls.

The foundation is laid for advanced features like indoor floor-plan navigation and Augmented Reality (AR) wayfinding, which can be integrated into the existing map logic.

Proposed integration with the academic calendar API would allow the "Smart Status" system to automatically adjust for holidays and exam periods.

v. Sustainable Development Goals (SDGs)

The USIU Campus Navigator directly supports key Sustainable Development Goals by leveraging technology to create an inclusive, efficient, and safe educational environment. By aligning with these goals, the system contributes meaningfully to the digital transformation of the university. Below are the three most relevant SDGs addressed by the project:

a) SDG 4: Quality Education

Provides a supportive learning environment by ensuring students can locate classrooms and academic resources (libraries, labs) quickly and efficiently.

Minimizes lateness and disruptions caused by navigational confusion, allowing students to focus on their academic pursuits.

Demonstrates the practical application of computer science skills to solve local community problems.

b) SDG 9: Industry, Innovation, and Infrastructure

Represents a digital innovation in infrastructure management, moving away from static physical maps to dynamic, interactive mobile solutions.

Optimizes the utilization of campus facilities by making their locations and operating hours transparent and easily accessible to all.

c) SDG 11: Sustainable Cities and Communities

Enhances the inclusivity of the campus by providing a tool that assists visitors and new community members in navigating the environment safely and independently.

The integration of emergency services contributes to a safer, more resilient community space.

7.6 Conclusion

The testing and evaluation phases played a vital role in validating the functionality, reliability, and scalability of the USIU Campus Navigator. These processes ensured the system met its operational goals—streamlining campus navigation, enabling intelligent facility discovery, and supporting real-time status tracking for all user roles.

Multiple testing methodologies were applied across the system lifecycle, including unit testing for individual components, integration testing for Google Maps and local data interactions, system testing for end-to-end workflows, and user acceptance testing (UAT) to verify that the platform aligns with stakeholder expectations. Manual validations on physical devices helped simulate realistic user interactions and edge cases, such as navigating without internet access or switching rapidly between tabs.

Evaluation extended beyond correctness to cover performance, usability, and stability. Performance benchmarks confirmed that the application initializes the map and filters search results within acceptable time limits (under 2 seconds). Usability assessments confirmed that the interface is intuitive for new students and visitors alike. Data handling and persistence mechanisms were reviewed to ensure that user favorites are retained reliably across sessions without data loss.

Ultimately, this rigorous testing and evaluation ensured that the USIU Campus Navigator is not only functionally complete but also resilient, user-friendly, and maintainable—ready for real-world deployment on campus and future scalability.